# A Visual Overview of the *SafeBound* Toolchain Workflow

*SafeBound Toolchain* Demo

*SafeBound* toolchain facilitates simulation-based safety testing for ADS. *SafeBound* comprises eight modules: (i) Scenario Selection Module, (ii) Scenario Implementation Module, (iii) Scenario Configuration Module, (iv) Simulator and ADS Integration Module, (v) Scenario Execution Module, (vi) Data Collection and Preprocessing Module, (vii) Safety Evaluation Module, and (viii) Data Visualization Module. Each module corresponds to a specific stage of the scenario-based safety testing pipeline, collectively forming a unified testing toolchain for evaluating the safety performance of ADS. In this tutorial, we provide a demo for using the SafeBound toolchain. This demo provides detailed information on Section 5 of the article.

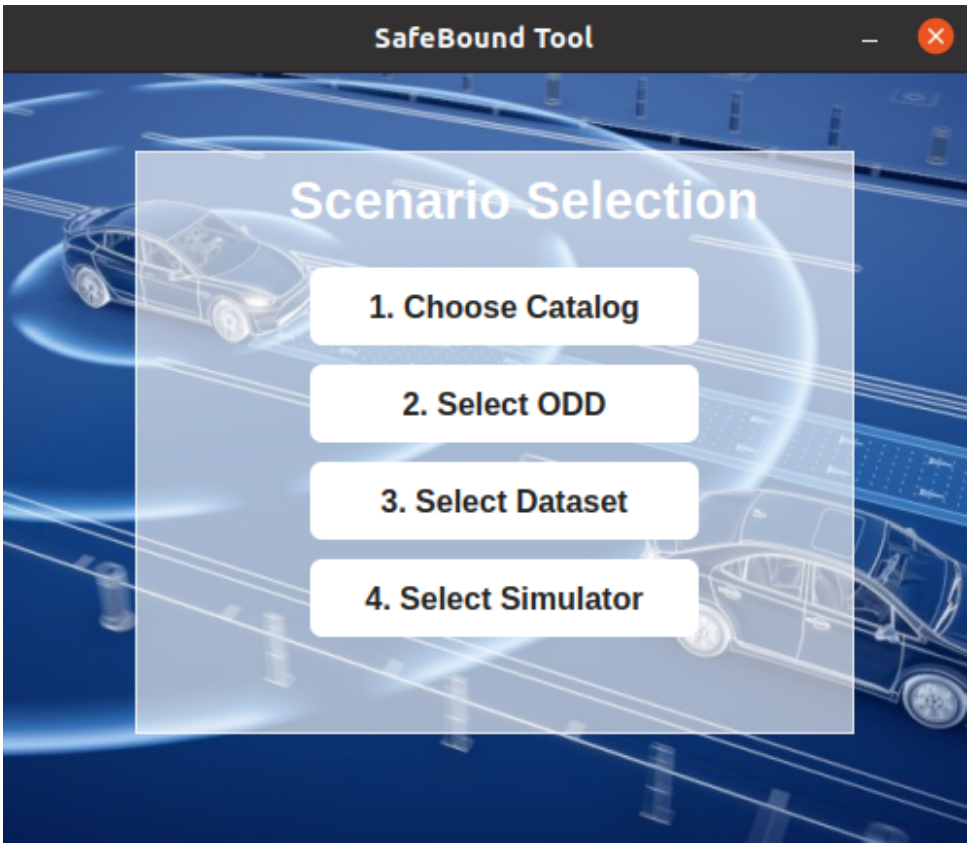



Figure 1: GUI for the Scenario Selection Module

## 1 Scenario Selection Module

For Scenario Selection Module, users provide four inputs.

**Step 1: Choose Catalog:** Select the appropriate scenario catalog. The current version includes two default catalogs: *Singapore* and *NHTSA*. For demonstration, the Singapore Catalog is selected. Users can also add new scenarios by clicking the *Add Scenario* button and selecting the catalog to which the scenario should be added.

Additionally, users have the option to create custom catalogs. For demonstration, we select the Singapore catalog.

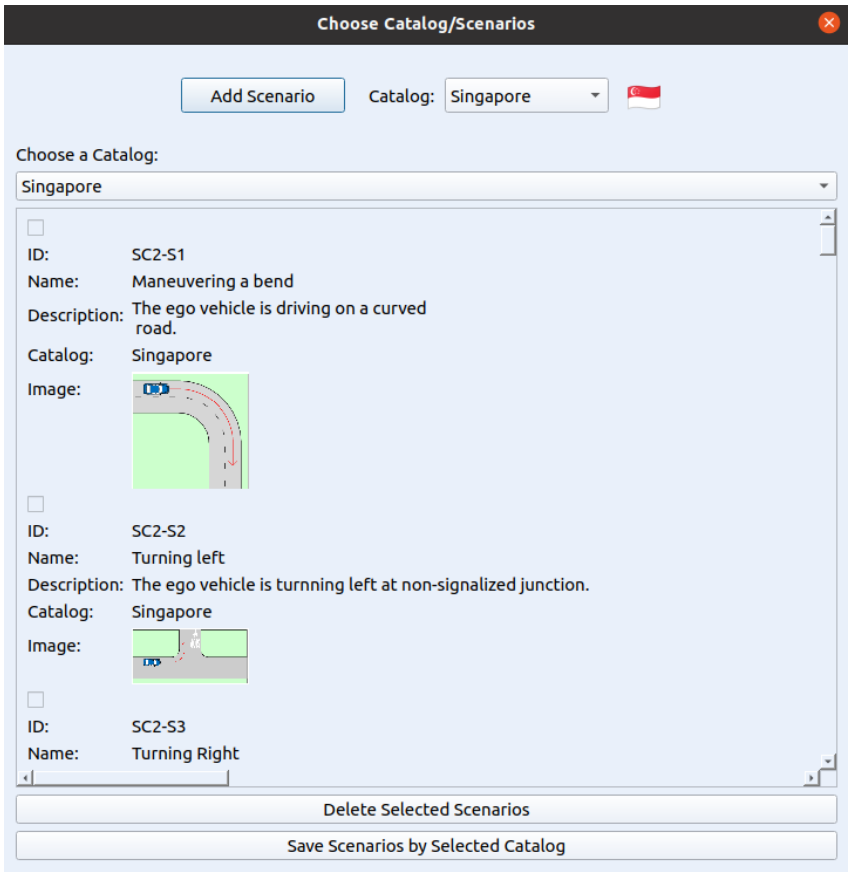



Figure 2: Choose Catalog window

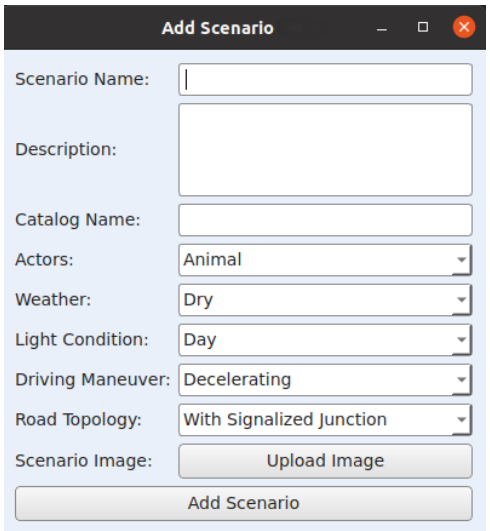
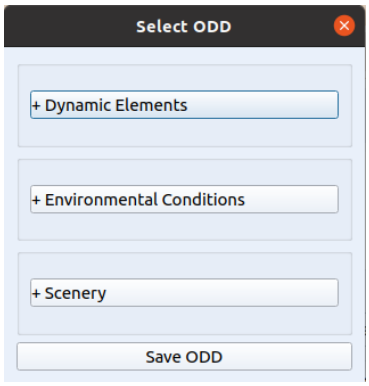


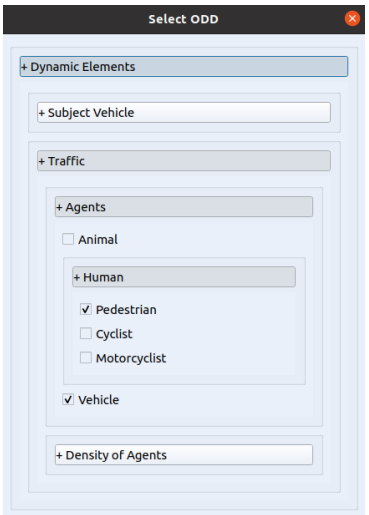

Figure 3: Add Scenario Interface

**Step 2: Select ODD:** Configures the Operational Design Domain (ODD). The ODD setup comprises three modules — *Dynamic Elements*, *Environment*, and *Scenery*. Each

tab can be expanded to specify ODD elements. Each tab in the ODD configuration can be expanded to define specific ODD elements. For the demonstration, we select the ODD parameters based on the operational capabilities of the UT-ADS.



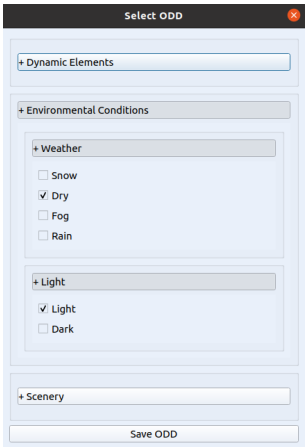

(a) ODD Input




(b) Dynamic Elements

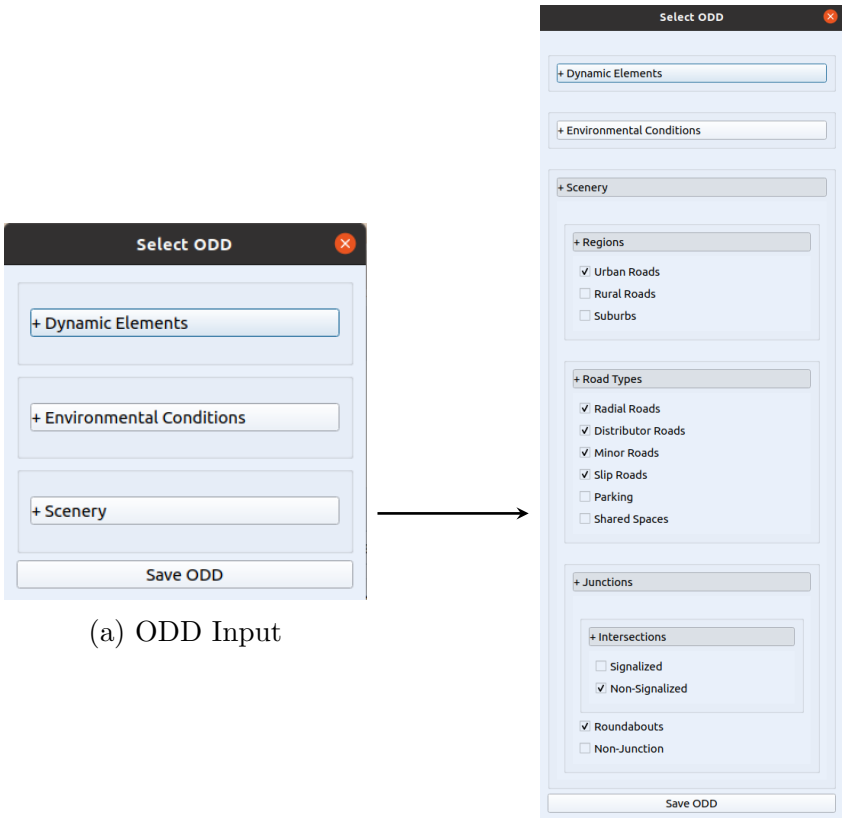
**Dynamic Elements:** For dynamic elements configuration, we select *pedestrians* and *vehicles*, as the UT-ADS is capable of detecting and interacting with both.

(c) ODD Input




(d) Environmental Conditions

**Environmental Conditions:** For environmental conditions configuration, we select *daytime* with *dry weather*.




(a) ODD Input

(b) Scenery Elements

**Scenery Elements:** For scenery elements configuration, we select an *urban road*. The road type is radial, including slip roads, minor roads, and distributed roads.

**Step 3: Select Dataset:** The tool allows users to select a dataset. The dataset is required to prioritize and select test scenarios. Currently, two datasets are supported. For demonstration, we select the US Accident Dataset.

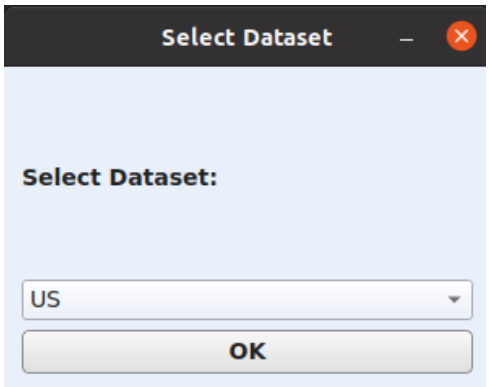



Figure 6: Dataset selection interface

**Step 4: Select Simulator:** The tool allows users to select a simulator. We select CARLA. The current version supports only CARLA.

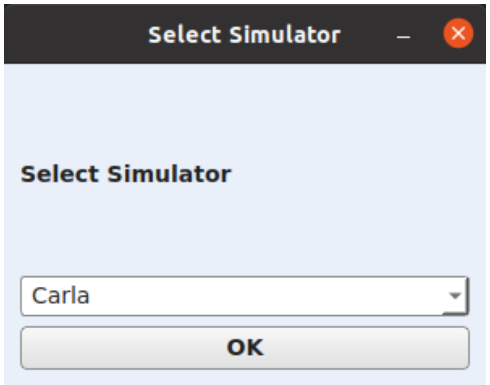



Figure 7: Simulator selection interface

**Final Output:** Upon giving four inputs, the tool generates a structured list of selected scenarios, including their IDs, scenario names, descriptions, scenario group, priority, and preview images.

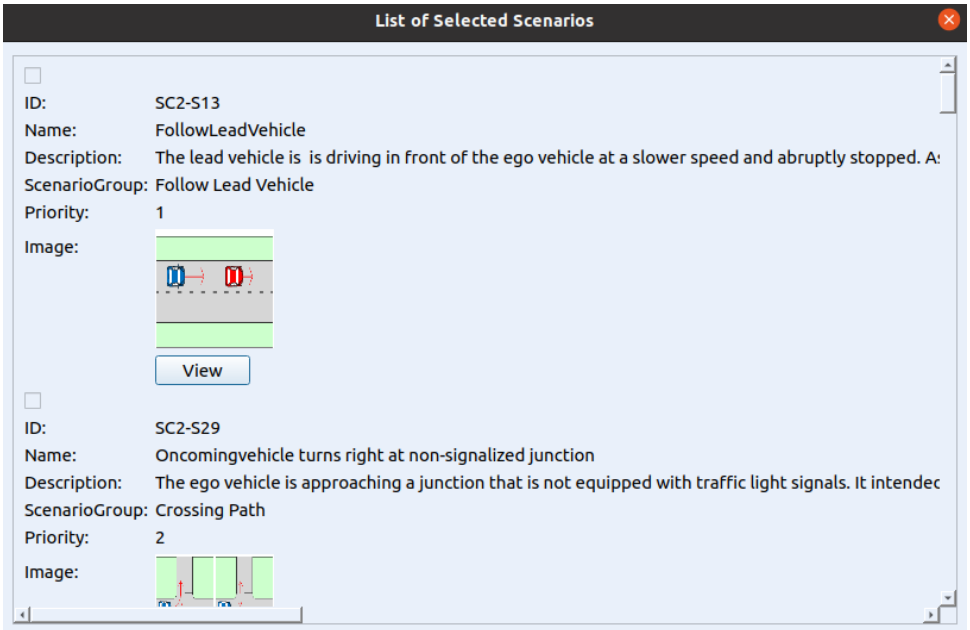



Figure 8: Final list of selected scenarios

# 2 Scenario Implementation Module

Once the scenarios are selected, the tool maps them to their corresponding OpenSCENARIO templates and converts them into executable Python scripts. For demonstration, we select the top-priority scenario. The top-prioritized scenario is Follow Lead Vehicle; the tool generates the corresponding executable file, `FollowLeadVehicle.py`. No user interaction is required for this step.

# 3 Scenario Configuration

Each scenario includes a *View* button. Selecting *View* opens a configuration window where users can input scenario parameters. For demonstration, we configure the parameter shown in the figure below.

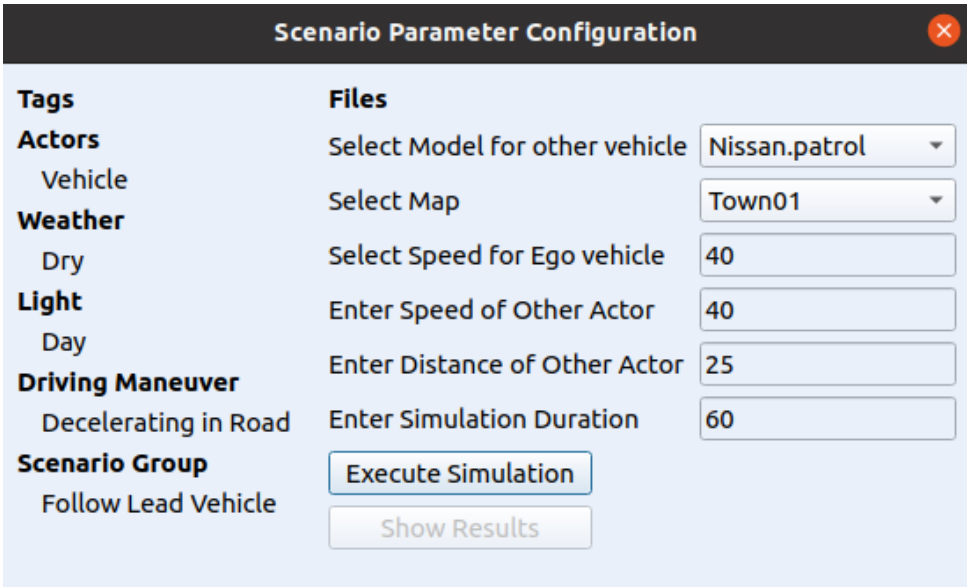



Figure 9: Scenario parameter configuration interface

Once the parameters are configured, the tool generates `FollowLeadVehicle.xml` files for the simulator and updates the `FollowLeadVehicle.py` with user-configured parameters.

# 4 Simulator and ADS Integration Module

Upon clicking on the *Execute Simulation* button, CARLA is launched first, followed by the UT-ADS stack and subsequently the scenario runner. The environment is set up, and the actors are initialized.

# 5 Scenario Execution

The scenario starts execution on the highlighted path in green. The system visualizes both the CARLA simulation view and the RViz perception view.

(a) CARLA view (b) RViz view

Figure 10: Scenario execution in CARLA and RViz

After the simulation completes, the system automatically generates the log file `FollowLeadVehicle.log`. No user interaction is required for this step..

# 6 Data Collection and Preprocessing

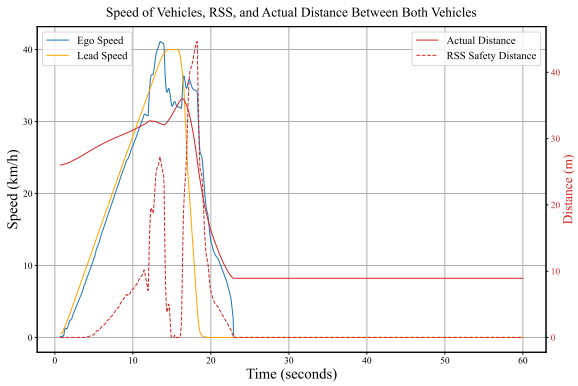
Once scenario execution finishes, the system processes the generated `FollowLeadVehicle.log` file and converts it into `FollowLeadVehicle_data.csv`. During this stage, invalid or incomplete records are filtered out to ensure data consistency.

# 7 Safety Evaluation Module

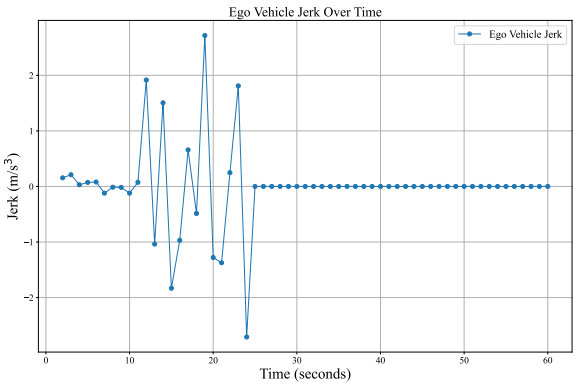
The system then computes three safety metrics (TTC, RSS safe distance, and jerk) and stores the results in `FollowLeadVehicle_metrics.csv`. No user interaction is required for this step.

# 8 Data Visualization

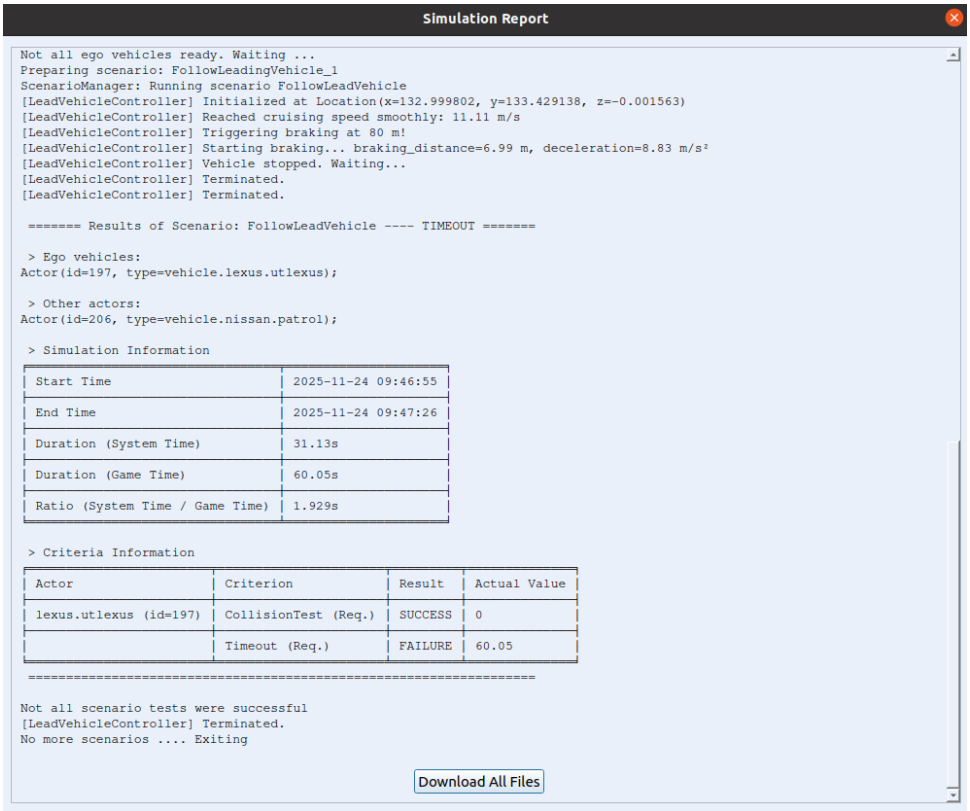
After the safety metrics are computed, selecting *Show Results* displays the corresponding graphs and summary report as shown in Figure 11.




(a) RSS Minimum Safe Distance




(b) Jerk



Simulation Report

```
Not all ego vehicles ready. Waiting ...
Preparing scenario: FollowLeadingVehicle_1
ScenarioManager: Running scenario FollowLeadVehicle
[LeadVehicleController] Initialized at Location(x=132.999802, y=133.429138, z=-0.001563)
[LeadVehicleController] Reached cruising speed smoothly: 11.11 m/s
[LeadVehicleController] Triggering braking at 80 m!
[LeadVehicleController] Starting braking... braking_distance=6.99 m, deceleration=8.83 m/s²
[LeadVehicleController] Vehicle stopped. Waiting...
[LeadVehicleController] Terminated.
[LeadVehicleController] Terminated.

 ======= Results of Scenario: FollowLeadVehicle ---- TIMEOUT =======

 > Ego vehicles:
Actor(id=197, type=vehicle.lexus.utlexus);

 > Other actors:
Actor(id=206, type=vehicle.nissan.patrol);

 > Simulation Information
```

| | |
|---|---|
| Start Time | 2025-11-24 09:46:55 |
| End Time | 2025-11-24 09:47:26 |
| Duration (System Time) | 31.13s |
| Duration (Game Time) | 60.05s |
| Ratio (System Time / Game Time) | 1.929s |

```
 > Criteria Information
```

| Actor | Criterion | Result | Actual Value |
|---|---|---|---|
| lexus.utlexus (id=197) | CollisionTest (Req.) | SUCCESS | 0 |
| | Timeout (Req.) | FAILURE | 60.05 |

```
 ==================================================================

Not all scenario tests were successful
[LeadVehicleController] Terminated.
No more scenarios .... Exiting
```

Download All Files

(c) Simulation Report

Figure 11: Visualization of simulation results for a Manual Configuration showing (a) RSS safe distance, (b) Jerk, (Simulation Report)